

Cambridge Computer Laboratory Systems Research Group

Forkable Sandboxes

The Runtime Layer for
AI Software Factories

Yossi Eliaz

Principal Engineer, Incredibuild / islo.dev

Lecturer, Holon Institute of Technology

15 October 2026 FW11 + Microsoft Teams

A software factory is a search loop over executable changes

Here, a software factory repeatedly proposes, runs, evaluates, and decides whether to promote a change.

propose $\longrightarrow$ execute $\longrightarrow$ evaluate $\longrightarrow$ promote or revise

- An agent changes files, processes, builds, and sometimes services.
- Tests and traces influence the next candidate.
- Each trial inherits some computer state from the previous one.

Execution state is part of the search algorithm's working state.

Fork the machine. What did we actually duplicate?

Thought experiment: one parser failure, eight proposed patches, and one prepared development machine.

one reached state S_0 $\longrightarrow$ eight isolated continuations

1. Did each patch run from the same relevant state after its own rebuild?
2. Do eight passing runs give eight independent pieces of evidence?
3. Can a child alter the verifier or publish its patch before review?

A fork creates possible futures; state, evidence, and authority still need contracts.

The question is whether a useful state can be branched

Start from an identified reached state S_0 and run alternative continuations.

Falsifiable systems hypothesis

For workloads with an expensive shared prefix, restoring or forking a prepared machine can lower total search cost relative to rebuilding that prefix for every candidate, while preserving the declared continuation behavior.

- **Proposed:** a continuation and authority contract for forkable search.
- **Implemented:** a reference evidence reducer and one named-snapshot worker trace.
- **Unmeasured:** a controlled end-to-end runtime speedup or general dependence model.

The benefit is workload-dependent; the contract is the research object.

A workload contract says what may change

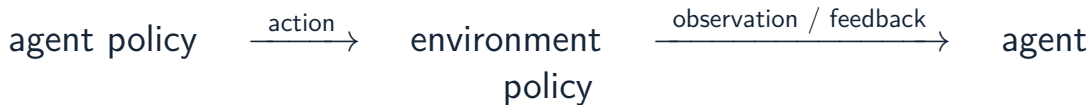
A benchmark needs a fixed task and a declared execution boundary before it can compare runtimes.

Contract field	Question to answer
Task and objective	What outcome counts as success?
Initial state	Which repository, services, fixtures, and versions define S_0 ?
Allowed actions	Which files, processes, tools, devices, and network calls may change?
Observation boundary	What can the agent and evaluator see?
Budget and stopping	Which CPU, GPU, memory, time, and model costs count?

A reproducible task is an executable specification, not just a prompt.

An environment is the interaction contract

An environment defines observations, actions, state transitions, feedback, and termination.



Environment examples

- Terminal or browser
- API or workflow
- Game or simulator
- Robot interface

Sandbox role

- Contains some executable state
- Restricts access and effects
- May implement one environment

A sandbox boundary and an environment's task semantics are separate design choices.

A coding episode includes its reset semantics

For coding RL, the environment is part of the training example.

$$\text{episode} = (S_0, \text{task}, \text{tools}, \text{verifier}, \text{reset}, \text{budget})$$

- A rollout starts from a particular repository and service state.
- The policy produces actions; the environment returns observations and outcomes.
- The learner uses verified outcomes to update policy parameters.

prepare $S_0 \rightarrow$ run a policy $\rightarrow$ verify outside the worker $\rightarrow$ retain trajectory $\rightarrow$ reset
or restore

Changing reset changes the distribution of experience available to learning.

Learned worlds and executable worlds reduce different costs

A world model predicts possible futures; a fork runs a continuation of captured executable state.

	Learned world model	Forked executable state
Future	Predicted transitions in a learned representation.	Actual program transitions within the captured boundary.
Main cost reduced	Interaction with a slow or scarce environment.	Rebuilding a costly software prefix.
Main limitation	Model error and out-of-distribution behavior.	Capture fidelity and omitted external state.

Models make imagined rollouts cheap; forks make executable rollouts cheap.

A fork runs the actual program only inside the boundary it captures.

Continuation state has several parts

A transcript records observations; it does not necessarily record the machine needed to continue.

Environment state S_t

- Files, processes, memory, and local services
- Databases, clocks, random streams, pending I/O
- Device state supported by the runtime

Agent and external state

- Agent history, policy version, sampling state
- Remote APIs, credentials, accounts, other users
- Evaluator state and objective version

$$o_t = \text{observe}(S_t), \quad S_{t+1} \sim P(\cdot \mid S_t, a_t; X_t)$$

The runtime can capture only a declared subset of the interacting system.

Restore fidelity is a testable observation claim

A snapshot is useful when its continuation preserves the behavior relevant to a declared task.

$$\forall \pi \in \Pi : \quad D_{\mathcal{O}}(\mathcal{L}_{0:h}^{\text{direct}}(\pi, X), \mathcal{L}_{0:h}^{\text{restore}}(\pi, X)) \leq \varepsilon$$

- Start from the same reached state and agent history; fix or mediate external-state policy X .
- Declare observable set $\mathcal{O}$, horizon h , policy class Π , distance $D_{\mathcal{O}}$, and tolerance ε .
- Use exact selected outputs for deterministic tests; compare distributions for stochastic tasks.

A successful restore call is not itself a fidelity result.

The snapshot boundary ends at real external effects

A local restore cannot undo an action already accepted by another system.

State	Possible mechanism	Residual risk
Local process and files	Process or VM checkpoint.	Unsupported devices and in-flight I/O.
Remote database or API	Private fixture, proxy, or record/replay.	Remote writes may persist after local rollback.
Credentials and identity	Short-lived, scoped lease minted per child.	Copied ambient secrets outlive intended authority.
Robot, assay, wall clock	Simulator, digital twin, or controlled interface.	Physical effects and time cannot be rewound.

State the boundary before claiming reproducibility or counterfactual control.

A parser experiment needs a real branch lifecycle

Fork after reproducing the failure; apply each patch and build the candidate inside its child.

1. Prepare a pinned repository, dependencies, services, and test fixtures.
2. Reproduce the parser failure and checkpoint the declared state S_0 .
3. In each child, apply one candidate change, rebuild or reload the parser, then run the same tests.
4. Send artifacts and receipts to an evaluator outside the child; discard the worker.

Important boundary

Changing a source file does not change code already loaded in a running process. Reuse the preparation; rebuild the candidate that changed.

A fork saves only the work that remains valid after the intervention.

Separate reusable preparation from candidate work

The system should identify which layers survive a proposed change.

Layer	Reuse policy	Validation
Repository and dependencies	Pin by content digest.	Confirm candidate starts from the intended parent.
Build artifacts	Share read-only within a trust domain.	Key by inputs; invalidate on relevant change.
Process and service state	Restore when continuation is supported.	Compare required observations and state digests.
Secrets and remote state	Reissue capabilities; proxy external calls.	Audit effects outside the child.

A warm build factory can sit beside the isolation boundary.

The break-even point depends on the shared prefix

An additive model compares total execution work, not parallel wall time.

$$C_{\text{replay}} = N(C_p + C_s), \quad C_{\text{fork}} = C_p + C_c + N(C_f + C_s)$$

- N : alternatives; C_p : rebuilding the shared prefix.
- C_c : capturing state; C_f : restoring or forking each child.
- C_s : candidate patch, rebuild, execution, and scoring.

branching saves work in this model if $(N - 1)C_p > C_c + NC_f$

For the parser campaign, measure the reusable prefix and each child's divergence; wall time also depends on contention.

Branching spans several systems mechanisms

A fork may name a filesystem layer, a process, a VM, or an execution trace.

Mechanism	State it can expose	Typical design question
Worktree or filesystem layer	Repository files and artifacts.	Are process and service states rebuilt?
Process copy-on-write	Address space and process creation.	Which kernel objects or peers remain shared?
Container checkpoint	Supported process tree and namespaces.	Which devices, networks, and volumes are captured?
MicroVM snapshot	Guest CPU and memory plus supported devices.	How are block storage and remote dependencies coordinated?
Trace or reversible execution	Recorded effects and selected checkpoints.	Which effects can be replayed faithfully?

The correct mechanism follows the continuation contract.

Isolation thickness and branch cost are different axes

A low-latency fork is useful only if it contains the effects the task must contain.

Boundary	Useful property	Question for an untrusted agent
Process or namespace	Low overhead and familiar OS sharing.	Can the worker reach sensitive host resources?
User-space kernel or sandbox	Mediated system calls and policy control.	Which kernel and device behaviors are supported?
MicroVM	Separate guest kernel and stronger isolation boundary.	What is the density and snapshot cost for this workload?

Isolation, density, state scope, and fork latency must be measured separately.

Agache et al., Firecracker, NSDI 2020.

Published systems show that fast branching is feasible

Recent systems report substantially different operations and workload boundaries.

DeltaBox

- Authors report about 10.83 ms of checkpoint work.
- Fast template restore is about 1.86 ms.
- Checkpoint work overlaps model inference in their design.

Shepherd

- Authors report 134–143 ms forks.
- Copy-on-write fork across images from 42 MB to 5.8 GB.
- The fork is a value in a reversible agent trace.

Interpretation

These are each paper's measurements. They use different operations and setups; they are not a cross-paper ranking or a result from my implementation.

Dong et al., DeltaBox (2026); Yu et al., Shepherd (2026).

Copy-on-write moves cost to divergence

At fork time, unchanged pages can remain shared; writes create branch-specific state.

$$M_{\text{total}} \approx M_{\text{shared}} + \sum_{i=1}^N M_{\text{private},i}$$

- Read-heavy children may preserve much of the shared working set.
- Write-heavy children pay through page faults, copied pages, and storage overlays.
- A fork can be cheap to create while later writes become expensive.

Measure peak memory, write amplification, and useful-work latency after the fork.

Credentials and network identity need their own policy

Copying a machine can also copy ambient authority unless the runtime changes it.

- Give each child a scoped capability for the specific task.
- Mint short-lived credentials outside the snapshot.
- Route network calls through a broker when writes must be logged or denied.
- Treat a remote effect as durable even if the local child is later destroyed.

Example

A branch may read a private test fixture but may not publish a package or merge a patch.

Fork state deliberately; grant capabilities deliberately.

Warmth is useful when its sharing scope is explicit

Build reuse can accelerate branches while creating a channel between them.

Resource	Reuse policy	Failure to consider
Immutable dependency image	Share by digest.	Stale or poisoned base image.
Build cache	Share within a trust domain.	Cross-tenant timing or data leakage.
Compiler and package downloads	Prewarm outside child state.	External state changes during the campaign.
Writable scratch space	Keep private to the child.	Hidden cross-branch contamination.

The warm factory belongs beside the fork, with an explicit trust boundary.

A crash should destroy a worker, not the experiment

Keep durable task identity outside disposable execution.

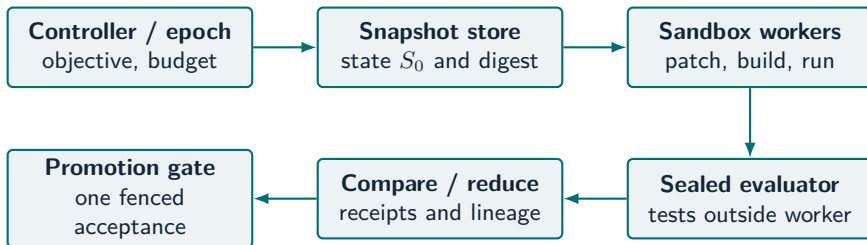
campaign $\rightarrow$ epoch $\rightarrow$ worker $\rightarrow$ receipt

- An epoch pins parent state, objective, evaluator, budget, and task data.
- A failed worker can be replaced under the same declared contract.
- A changed objective or fixture starts a new epoch.
- The receipt outlives the machine that produced it.

Recovery must not silently change what is being evaluated.

The controller, runtime, and evaluator have different jobs

A proposed architecture makes the state path and the authority path visible.



Exercised: snapshot $\rightarrow$ worker $\rightarrow$ reducer. **Proposed:** sealed evaluator and promotion gate.

A parser campaign gives the contract an API shape

The operation sequence makes reusable state, candidate work, and authority explicit.

1. `checkpoint(prepared_parser)` records a named state and parent digest.
2. For each patch Δ_i : `fork(snapshot, scoped_lease)`, then `apply`, `rebuild`, and `run` it.
3. `evaluate(artifact, verifier_digest)` runs outside the child and seals a receipt.
4. `select(receipts)` compares candidates; `reduce(results)` pools statistical evidence only when its assumptions hold.
5. `promote(epoch, accepted_receipt)` admits one result through a fenced gate.

The restore–run–capture–reduce path has one integration trace; this full API is a research contract.

Stochastic hill climbing uses rollback as an experiment primitive

One candidate changes one measured property; a controller decides whether to keep it.

current candidate → sample change → fork and run → score against fixed harness
→ keep, revert, or retry

- Freeze the objective and evaluation procedure.
- Record unsuccessful trials as well as accepted changes.
- Repeat when the score is noisy.

A cheap revert encourages exploration; it does not make one score reliable.

Genetic search branches candidate lineages

A population search mutates or recombines artifacts, then evaluates each candidate in a controlled environment.

population → select parents → mutate / crossover



isolated child → evaluate → update archive

- The genotype may be code, a harness, a policy, or runtime configuration.
- Machine-state lineage and candidate-code lineage are different graphs.
- Preserve diversity when multiple ideas may be useful stepping stones.

Crossover recombines candidate representations; it does not merge live machines.

Jeddi et al., GEAR: Genetic AutoResearch for Agentic Code Evolution (2026).

Automated research searches over experiments too

The object being changed may be the program, the hypothesis, or the next measurement.

Search object	Example proposal	Evidence returned
Program	Change a parser or optimizer.	Tests, runtime, memory, artifacts.
Policy	Change actions or sampling strategy.	Rollouts and verified reward.
Experiment	Change workload slice or parameter.	Measurement and uncertainty.
Runtime policy	Change fork point or cache scope.	Fidelity, cost, and leakage results.

Keep the study objective versioned; log changes to the measurement protocol as proposals.

RL consumes branches as trajectories

An RL worker executes actions; a learner uses verified experience to update a policy.

policy $\pi_\theta \rightarrow$ environment trajectory $\rightarrow$ reward / verifier $\rightarrow$ learner update $\rightarrow$ policy $\pi_{\theta'}$

- A fork can reduce the cost of constructing repeated episode state.
- The rollout policy and reset distribution determine the data collected.
- Total inference, environment, verifier, and runtime costs count.

A sandbox speedup is useful only if it improves the full training loop under a fixed budget.

Run and evaluation need separate write permissions

A searcher must not be able to rewrite the test that declares it successful.

RUN

- Candidate code and tools
- Writable task workspace
- Scoped network capabilities

EVAL

- Pinned objective and verifier
- Protected parser regression fixtures
- Independent artifact inspection

RUN $\neq$ EVAL

Isolation protects the evaluator; task design determines whether its score is meaningful.

Wu et al., *From Rebound to Remedy* (2026).

The same runtime shape supports different workloads

The proposal, executable world, oracle, and scarce decision differ by domain.

Workload	What branches explore	What remains scarce
Software	Patches, tests, and agent tool paths.	Trusted merge decision.
RL	Rollouts, policies, and episode states.	Valid reward and checkpoint selection.
Simulation / robot	Parameters, controllers, and simulated conditions.	Hardware time and safe physical trials.
Computational biology	Model settings, sequences, and analysis pipelines.	Independent experimental validation.

The runtime pattern transfers; environment boundaries and scientific oracles remain domain-specific.

Computational biology makes the model boundary visible

In an actomyosin network model, local interaction rules can produce different large-scale structures.

- A computational branch can vary a parameter, initial condition, or model version.
- Outcomes can include connectivity, clustering, or predicted network morphology.
- A simulation score tests model consequences, not biology itself.
- Experiments are needed to validate whether the modeled mechanism fits biology.

Fork simulation state to explore model consequences; keep experimental claims tied to external validation.

Eliaz et al., [arXiv:2006.06503](https://arxiv.org/abs/2006.06503) (2020).

Shared error limits what repeated runs establish

Suppose K estimates have equal variance σ^2 and common pairwise error correlation ρ .

$$\text{Var}(\bar{X}) = \sigma^2 \left(\rho + \frac{1 - \rho}{K} \right)$$

- At fixed $\rho > 0$, increasing K cannot remove the shared-error floor.
- Fork lineage identifies common ancestry; it does not reveal the covariance.
- New random continuations can add conditional information when the experimental design justifies it.

Branch count alone does not tell us the effective amount of evidence.

Four processes can replay one test set

Imagine four retries of one parser candidate against the same 100 test cases.

4 isolated runs $\longrightarrow$ 4 result files $\longrightarrow$ 100 distinct test IDs

- A new worker ID does not turn reused tests into 400 independent cases.
- Exact evidence-ID overlap can be detected; shared model error and adaptive selection are harder.
- Distinct, designed rollouts may provide new conditional evidence.

Execution identity, evidence identity, and statistical independence are different properties.

A worker should return evidence with its result

A scalar score loses the information needed to interpret repeated or related work.

$$r_k = (\hat{\theta}_k, J_k, n_k, \mathcal{E}_k, \mathcal{L}_k, m_k)$$

- Estimate $\hat{\theta}_k$, information J_k , and sample count n_k .
- Evidence identifiers $\mathcal{E}_k$ and fork lineage $\mathcal{L}_k$.
- Execution metadata m_k : objective, model, task, verifier, and runtime versions.

The numeric summary and its provenance need different merge rules.

Eliaz, Evidence-Aware Reduction for Forkable Compute (2026).

A reducer must state its statistical assumptions

For a common target and independent evidence, per-observation information can be pooled.

$$P_k = n_k J_k, \quad P = \sum_k P_k, \quad \hat{\theta} = P^{-1} \sum_k P_k \hat{\theta}_k$$

- J_k is calibrated per-observation information; n_k is a literal sample count.
- Reject exact overlap of non-empty evidence IDs; preserve lineage and selection history.
- Disjoint IDs do not remove latent dependence or bias from adaptive selection.
- If dependence is unresolved, abstain from an added-precision claim.

The reference reducer implements this independent-evidence case; lineage alone does not supply covariance.

The current artifact has bounded, testable results

The paper separates algebra checks, synthetic behavior, and an execution trace.

Check	Observation	What it supports
Gaussian reference reducer	Flat and tree reductions match closed form.	Numeric implementation follows the stated algebra.
Unequal logistic shards	Pooled gap 0.0083 ± 0.0042 ; equal-shard average 0.177 ± 0.090 .	Information weighting helps in this synthetic eight-seed check.
Forged precision stress	Unprotected estimate 17.0004; heuristic output 4.9566.	One attack example and a heuristic response, not Byzantine robustness.

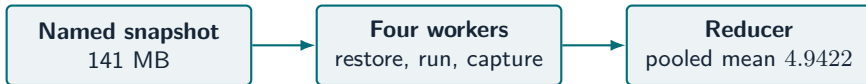
Logistic gap = distance to centralized MLE; $\pm$ = SD over eight fixed seeds.
These synthetic checks do not establish repeated-sampling statistical efficiency.

These results validate a reference artifact, not a universal agent-search benefit.

Eliaz (2026), evaluation in Evidence-Aware Reduction for Forkable Compute.

One trace exercises the snapshot-to-reducer path

The paper reports one named-snapshot integration run, not a speedup benchmark.



- The four concurrent worker round trips completed in **6.70 s client-observed total**.
- The seed-pinned full-sample mean was 4.9450.

Scope of evidence

No per-phase timing, cold-build baseline, or controlled cross-provider comparison is reported.

The end-to-end path runs; comparative runtime benefit remains an open experiment.

[Eliaz \(2026\)](#), named-snapshot integration trace.

Promotion is a separate, fenced operation

Workers produce artifacts and evidence; a controller decides whether either can become authoritative.

Invariant	Controller records	Worker cannot change
Parent identity	Immutable snapshot or artifact digest.	The campaign's trusted parent.
Objective identity	Task and objective digest.	The scoring rule for its own result.
Evaluation identity	Verifier version and evidence manifest.	Hidden parser tests or admission records.
Promotion identity	Current epoch and one acceptance token.	Merge, release, or policy checkpoint authority.

Durable intent, disposable execution, singular authority, fenced promotion.

The benchmark that would falsify the runtime hypothesis

Future test: fix task distribution, evaluator, policy, and total resource budget across paths.

Path	What it tests	Measure
Cold build	Reconstruct every episode.	Time to ready and total resource use.
Cached restart	Reuse immutable prepared artifacts.	Cache benefit and invalidation correctness.
Checkpoint restore	Resume supported state.	Restore fidelity and restart cost.
Forked fan-out	Branch from one reached parent.	Throughput, memory growth, and leakage.

Vary shared-prefix cost, branch write fraction, and fan-out. Check the declared observation-fidelity criterion; report task quality, CPU/GPU work, wall time, failures, and external effects.

The outcome is useful task progress per total compute under the same correctness contract.

The research agenda sits at the runtime–search boundary

The unresolved problems connect operating systems, learning, and statistical evidence.

1. **Continuation:** what is the minimal useful state across files, memory, GPU, sockets, and time?
2. **Capabilities:** how can credentials be reminted at fork speed without leaking identity?
3. **Dependence:** how should shared ancestry and adaptive selection affect a reducer?
4. **Scheduling:** when should a controller fork, replay, use a learned model, or stop?
5. **CoW channels:** which shared warmth creates a timing or residency side channel?

The runtime and the search policy should be evaluated together.

Forks expose futures; they do not certify them

A useful runtime can make controlled continuations cheaper while keeping evidence and authority explicit.

- Capture and restore a declared executable state.
- Isolate branches and account for external effects.
- Keep evaluator identity and promotion authority outside the worker.
- Carry lineage so a reducer can state what its result assumes.

Questions for Cambridge

Which state must a forkable machine preserve?

How should a system infer dependence between branches?

Where should warm state live so it does not become a channel?

Fork the machine. Keep the evidence. Promote through one explicit gate.